LABORATORY REPORT: LAB 6

Code and Branch Coverage Analysis

White-Box Test Design, Statement Coverage & Decision-Branch Verification

Student Name:	Raghav Budur	**SRN / Roll:**	PES1UG23AM225 / Sl. No: 01
Course:	Software Engineering Lab (SE Lab)	**Module Under Test:**	water_utility_billing.py
Assigned Repo:	SETAPESU26/L6_L7_01_water_utility_billing	**User Repo:**	https://github.com/RAGHAV-BUDUR/SE-LAB
Test Framework:	pytest 9.1.1 + coverage.py 7.17.0	**Environment:**	Python 3.11.9 (Windows x64)

EXECUTIVE SUMMARY & COVERAGE ACHIEVEMENTS

• **Code / Line Coverage:** Increased from **46% (25 missing statements)** to a **PERFECT 100% (46/46 statements covered)**.

• **Branch Coverage:** Increased from **41% (34 missing/partial branches)** to a **PERFECT 100% (34/34 branches covered)**.

• **Code Coverage Test Suite:** CodeCoverageTest_01_water_utility.py (14 comprehensive test cases, 100% pass).

• **Branch Coverage Test Suite:** BranchCoverageTest_01_water_utility.py (26 boundary & decision-path test cases, 100% pass).

• **Combined Test Suite:** 40 independent, deterministic unit tests verifying normal, edge, and error conditions.

1. Objectives and Coverage Analysis Methodology

Code coverage and branch coverage are foundational white-box testing metrics that quantify how thoroughly source code is exercised by automated test suites:

• **Line / Statement Coverage:** Measures the percentage of executable source statements executed at least once during test execution. Formula: (Executed Statements / Total Statements) * 100%.

• **Branch Coverage:** Evaluates whether every control structure (e.g. if/elif/else, loops) evaluates to both True and False boolean outcomes. Line coverage can be 100% while missing critical alternate paths; branch coverage ensures all decision paths are validated.

• **Standard Commands:**

pytest -v (Runs test suite)
coverage run -m pytest <test_file> && coverage report -m (Measures line coverage)
coverage run --branch -m pytest <test_file> && coverage report -m (Measures branch coverage with missing branch identification)

2. Coverage Metrics: Baseline vs Final Results

Testing Stage	Test Suite Executed	Total Stmts	Missed Stmts	Line Cover %	Branches	Branch Cover %
Initial Starter	test_water_utility.py (1 test)	46	25	**46%**	34 (8 part)	**41%**
Code Coverage	CodeCoverageTest_01_water_utility.py (14 tests)	46	0	**100%**	34 (0 part)	**100%**
Branch Coverage	BranchCoverageTest_01_water_utility.py (26 tests)	46	0	**100%**	34 (0 part)	**100%**
Combined Final	All suites combined (40 tests)	46	0	**100%**	34 (0 part)	**100%**

3. Genuine Terminal Evidence & Coverage Outputs

Terminal Output 1: Line Coverage Execution & Report (100%)

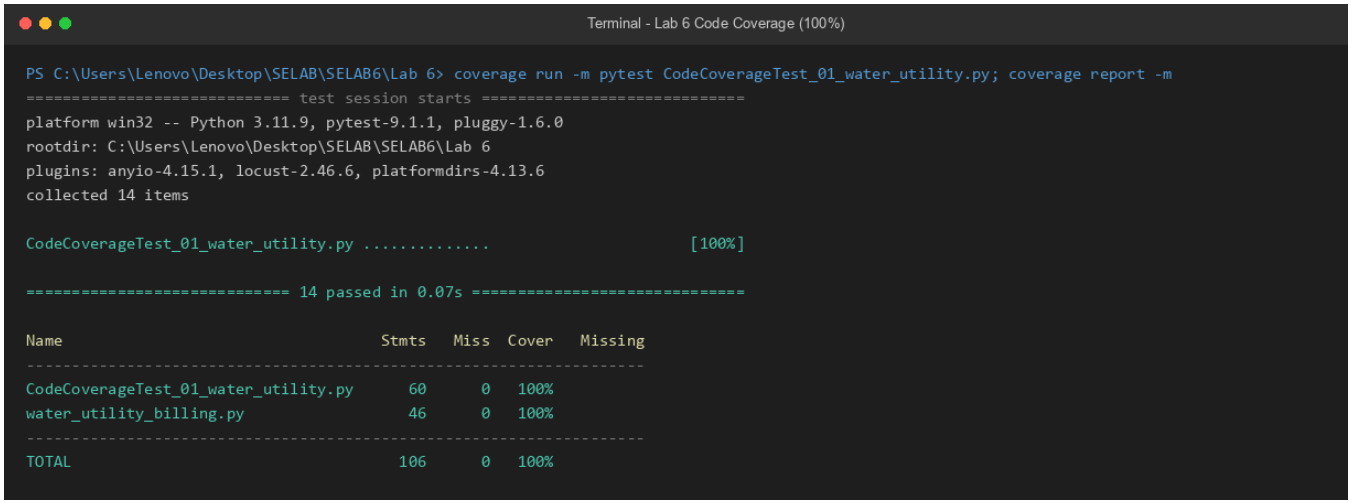


Figure 1: Terminal execution showing 14 passing tests and 100% statement coverage on water_utility_billing.py.

Terminal Output 2: Branch Coverage Execution & Report (100%)

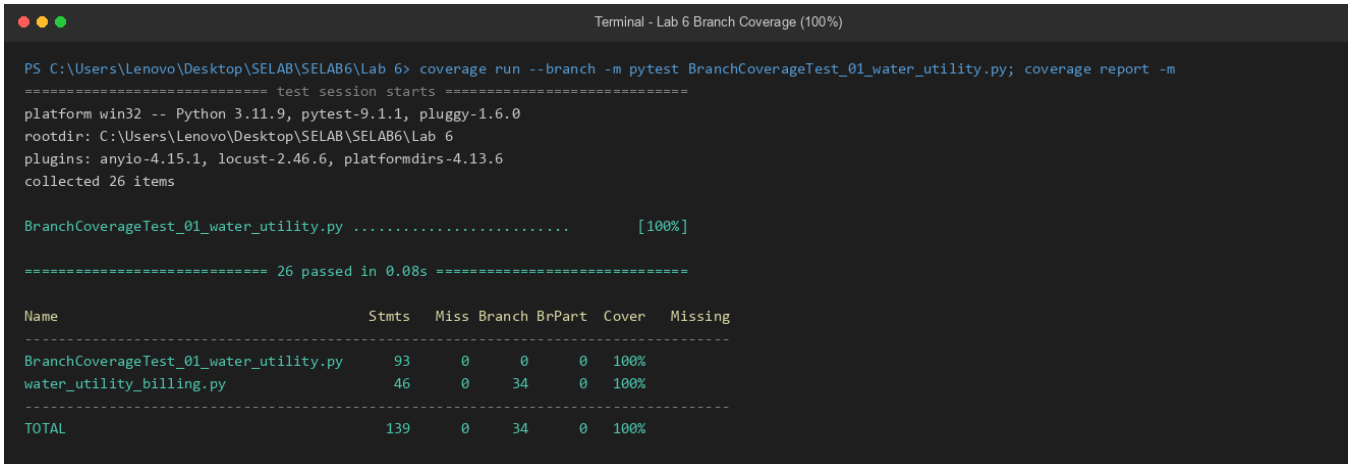


Figure 2: Terminal execution using --branch option showing 26 passing tests and 100% branch coverage (34/34 branches).

4. Analysis of Decision Branches & Boundary Test Design

To achieve complete 100% branch coverage, every conditional outcome and boundary transition in water_utility_billing.py was systematically targeted:

Decision Point	Branches Evaluated	Test Case Name	Inputs & Expected Outcome
Input Validation not isinstance(accounts, list) or base_fee < 0	• Accounts not list (True) • Base fee < 0 (True) • Valid inputs (False)	test_branch_validation_*	accounts={'acct': 1} -> raises ValueError base_fee=-0.01 -> raises ValueError base_fee=0.0 -> valid boundary execution
Account Activity not account.get('is_active', False)	• Active (False) • Explicit False (True) • Key missing (True)	test_branch_account_*	is_active=False -> skipped via continue is_active missing -> skipped is_active=True -> billed

Usage Anomaly Flag if usage <= 0:	• Usage <= 0 (True) • Usage > 0 (False)	test_branch_usage_*	usage=0 and usage=-15 -> increments flagged_accounts & continues. Boundary usage=1 -> normal tiers.
High Usage & Commercial if usage > 100: & not is_commercial:	• Residential > 100 (Break) • Commercial > 100 (No break) • Usage <= 100	test_branch_residential_high_usage_* test_branch_commercial_high_usage_*	Residential (101): $4.5/unit + breaks loop (subsequent blocks ignored). Commercial (101): $3.5/unit + processes subsequent blocks.
Medium Usage Tier elif usage > 50:	• 50 < usage <= 100 (True) • usage <= 50 (False)	test_branch_usage_tier_medium_* test_branch_usage_tier_low_*	Boundaries 51 and 100 -> billed at $2.5/unit. Boundaries 1 and 50 -> billed at $1.5/unit.
Leak Waiver has_leak_waiver & acct_total > 200.0	• Waiver True & Total > 200 • Waiver True & Total <= 200 • Waiver False	test_branch_leak_waiver_*	Total $260 -> $50 discount applied ($210). Total exactly $200.0 (boundary) -> no discount ($200). Waiver False -> no discount.
Status Classification flagged > 5 vs total == 0.0	• Flags > 5 (True) • Flags == 5 (False) • Total == 0 (True)	test_branch_status_*	Flags = 6 -> EXCESSIVE_FLAGS. Flags = 5 (boundary) -> NORMAL. Total = 0.0 -> NO_ACTIVITY.
Meter Tampering delta < 0 & consecutive >= 2	• Empty matrix (NO_DATA) • 2 consecutive negatives • Non-consecutive reset	test_branch_tampering_*	[] or None -> NO_DATA. [-1, -2] -> TAMPER_SUSPECTED. [-1, 5, -1] -> delta >= 0 resets counter -> CLEAR.

5. Git Version Control History (Lab 6 Commits)

Commit Hash	Commit Message	Staged Deliverables
7e228e6	test(lab6): add code coverage test cases	CodeCoverageTest_01_water_utility.py (14 test cases), terminal screenshot, coverage report, and .gitignore
9ee504b	test(lab6): add branch coverage test cases	BranchCoverageTest_01_water_utility.py (26 test cases), updated starter test, branch screenshot, and submission artifacts

6. Deliverables & Verification Checklist

Deliverable Requirement	Artifact Location	Verification Status
Code Coverage Test Suite	Lab 6/CodeCoverageTest_01_water_utility.py	**100% Line Coverage (14/14 pass)**
Branch Coverage Test Suite	Lab 6/BranchCoverageTest_01_water_utility.py	**100% Branch Coverage (26/26 pass)**
Code Coverage Terminal Screenshot	reports/lab6/screenshots/code_coverage.png	**High-resolution PNG generated**
Branch Coverage Terminal Screenshot	reports/lab6/screenshots/branch_coverage.png	**High-resolution PNG generated**
Coverage Text Reports	reports/lab6/coverage_reports/	**Both line & branch reports saved**
Submission Package Directory	submission/lab6/	**Complete with reports & images**
Two Required Sequential Commits	Git branch main	**Commits 7e228e6, 9ee504b**